Group 4 - CloudNine Systems

Milestone 1: Draft Analysis Report

Core design proposal: 23 tables. Updated October 13, 2026. Team roster and adoption remain pending.

1. Scenario and purpose

CloudNine Systems is a fictional provider of shared compute, storage and network services. Clients reserve capacity; staff allocate resources, track deployments and consumption, and bill responsible accounts. Our core database answers who uses which resources, what capacity remains available, and how costs are shared.

The main workflow is CLIENT -> RESERVATION -> DEPLOYMENT -> RESOURCE_ALLOCATION -> USAGE_RECORD -> CHARGE -> CHARGE_SHARE -> INVOICE. Intermediate service and invoice-line tables connect the detailed relationships shown in the ERD. Users and payers may belong to different clients.

2. Organizational schema

Clients and people (4): CLIENT, USER_ACCOUNT, CLIENT_MEMBERSHIP, PROJECT. Clients can be individuals, teams or organizations. Membership links people to clients; projects group deployments. Staff accounts are identified by account_kind and staff_role.

Infrastructure and availability (6): REGION, AVAILABILITY_ZONE, DATA_CENTER, RESOURCE_TYPE, RESOURCE, RESOURCE_STATE. Resources have scalar processing, memory, storage and network capacities. State intervals record operational availability, maintenance and failures.

Reservations and requests (2): RESERVATION, RESERVATION_ITEM. A reservation stores its current dates and approval; items specify resource types, quantities, minimum capacities and location preferences. Physical assignment may occur later.

Deployments and assignments (3): DEPLOYMENT, RESOURCE_ALLOCATION, DEPLOYMENT_SERVICE. A deployment has its current status, priority and scaling mode. Time-based allocations preserve which resources served it, including replacements. Provisioned services identify the billable offerings.

Services, rates and metering (3): SERVICE, SERVICE_RATE, USAGE_RECORD. Each service has one billing unit. Effective-dated prices and estimated or actual consumption support cost reporting.

Billing and invoices (5): BILLING_ACCOUNT, CHARGE, CHARGE_SHARE, INVOICE, INVOICE_LINE. Charges are assigned to one or more payer accounts. Invoice lines associate finalized shares with an invoice. Invoice totals are calculated, not stored redundantly.

3. Design decisions and 3NF

Separate demand from assignment: a request for two instances is distinct from the two resources actually assigned. Replacing a failed resource closes one allocation and creates another; it does not overwrite the original assignment. Maintenance state is separate from occupancy, which is derived from allocations.

Separate usage from billing: service consumption and effective prices produce charges. Staff enter actual monetary shares rather than maintaining a configurable cost-sharing engine. Each charge share stores only its charge, payer and amount; the former duplicated billing-rule reference has been removed.

1NF: columns hold scalar values. Client membership, reservation items, allocations and payer shares occupy separate rows. vCPU, memory GiB, storage GiB and network Gbps are distinct dimensions, not serialized lists. Notes are descriptive text, not hidden collections of entities.

2NF: each table has a single-column primary key, and all non-key attributes describe its identified record. Composite alternate keys are listed below. No partial dependency on an alternate key's component is assumed.

3NF: location follows RESOURCE -> DATA_CENTER -> AVAILABILITY_ZONE -> REGION; ancestor IDs are not repeated in RESOURCE. An item chooses at most one location level. Usage refers to a deployment service without repeating its deployment, service or billing unit. Rates are stored separately from service descriptions. Shares omit invoice membership, which belongs to INVOICE_LINE. Current totals and availability are derived.

Each CHARGE has one context: usage, direct service or an original charge. Metered rows have a unique populated usage_id, while manual charges do not copy a deployment or service derived from their context. Nullable unique usage_id is a conditional identifier for metered rows, not a candidate key for every row. These charge types are combined using explicit XOR constraints; if the course requires a null-free relation model, they can instead be split into typed relations during physical design. Final charge amounts are posted accounting facts, with later corrections represented by separate adjustments.

Under the stated business dependencies, the model is intended to satisfy 3NF. Keys alone do not prove normalization: additional dependencies identified during implementation require review. A staff_role value does not determine any other stored account attribute; account_kind is independently assigned.

4. Integrity rules and assumptions

R1 - Identity and scope. Each data center belongs to one zone, and each zone to one region. CLIENT has an optional single parent; hierarchy cycles are prohibited. Every deployment's optional project must belong to its reservation's client. Billing accounts may belong to a different client by agreement. Staff approvals and assignments must reference active staff or dual-role accounts; null assignment actors represent automated actions.

R2 - Time and resource identity. Use UTC and half-open intervals [start, end); an optional null end means open-ended. End must exceed start. Resources are independently allocatable assets, not both a cluster and its components. Capacity and resource type are immutable once allocations reference a resource. A changed configuration becomes a new resource version with a new local identifier, and assignments transfer to it. Retirement closes the old resource's usable state. Resource reconfiguration history and links between versions are outside scope. State intervals for the same resource cannot overlap; operational state and allocations may overlap.

R3 - Reservation fulfillment. Approved reservations contain at least one item. Each allocation's item and deployment must belong to the same reservation. Each resource must match the item's resource type. Allocated capacity (resource capacity times capacity_fraction) must meet each per-instance minimum. At any instant, planned or active allocation count across all deployments for an item cannot exceed its requested quantity; full fulfillment requires exactly that quantity of distinct resources. No two concurrent allocations for the same item may use the same resource. Partial fulfillment is visible as a shortage, not a fulfilled reservation.

R4 - Capacity and changes. capacity_fraction is greater than zero and at most one. Overlapping planned or active fractions cannot total more than one per resource. Maintenance, failure and retirement block usable assignments. Allocations must fit within the reservation dates, and active allocations within deployment operating dates. A replacement closes the old interval before the new one begins and records its predecessor, reason and assigning actor; predecessor links cannot cycle and must remain within the same deployment and item. Changes to current reservation dates, items or quantities must revalidate all allocations and retain compatibility with historical

assignments. Incompatible changes require a new reservation. Reservation revision history is deliberately omitted.

R5 - Locations and quantities. Each item selects at most one of region, zone or data center; ancestors are derived. Mandatory location constraints must be met; preference substitutions require an allocation reason. Counts are positive integers; capacities, consumption and rates are nonnegative. Each requested instance has the listed minimum capacities. A proportional allocation shares every resource dimension equally; independent multidimensional partitioning is excluded.

R6 - Metering and rates. Each usage record belongs to one deployment service. Resource-based usage also requires an allocation in the same deployment, covering its metered interval. All usage falls inside the service's provisioning interval. Each service has one immutable billing unit; a unit change creates a new offering. Standard rate intervals for a service cannot overlap. Split consumption at rate boundaries, and use the matching service rate. Estimates and actuals are distinct record types and are never summed together as actual use.

R7 - Corrections and charges. Usage corrections create a new row linked to the previous one. Links cannot cycle or branch, and retain source, interval and record type; only the latest actual row is used for current consumption totals. CHARGE fills exactly one of usage_id, deployment_service_id and adjustment_of_charge_id. Metered charges require a matching service_rate_id; direct surcharges and adjustments leave it null. A usage record can have at most one charge. Standard metered amount is quantity times unit_price, rounded to cents. Only actual usage can support a final metered charge; estimates support provisional charges that are voided when actual consumption is charged. Final charges are immutable. Correct posted consumption using a reversing adjustment and a replacement charge. Adjustment chains cannot cycle; credits are negative, and manual adjustments require staff authorization and a reason.

R8 - Sharing and invoicing. Staff record one or more CHARGE_SHARE rows for every final charge, with amounts totaling the charge. Credits have nonpositive shares; positive charges have nonnegative shares. Store actual money allocations, not a duplicated fraction or automatic rule. All money is USD with fixed precision. A share can belong to at most one invoice, whose account must match the share's payer. Only final charges can be invoiced; issued invoices contain at least one line. Invoice totals are derived from lines. Invoice status tracks issuance or voiding, not payment settlement.

R9 - Candidate keys. Additional uniqueness constraints are USER_ACCOUNT.email; REGION.name; RESOURCE_TYPE.name; SERVICE.name; AVAILABILITY_ZONE(region_id, name); DATA_CENTER(zone_id, name); RESOURCE(data_center_id, local_identifier); SERVICE_RATE(service_id, valid_from); CHARGE_SHARE(charge_id, billing_account_id); CHARGE.usage_id when populated; and USAGE_RECORD.supersedes_usage_id when populated. INVOICE_LINE.charge_share_id is both PK and FK, enforcing one invoice per share. Membership intervals for the same client, user and role cannot overlap.

PK, FK, UNIQUE and CHECK constraints enforce local rules. Capacity totals, mandatory children, interval conflicts, cross-table ownership and sharing totals require transactional checks or triggers. They are specified here, not yet implemented as a production database.

5. Coverage and deliberate simplifications

Core coverage: people and organizational membership; geographic infrastructure; resource types, tiers and availability; advance reservations and multiple requested types; deployment status, priorities, allocations and substitutions; services, rates, estimated and actual use; separate users and payers; cost sharing, surcharges, credits and invoices.

History: allocation intervals, availability, membership, service prices and usage corrections are retained. Reservation revisions and complete deployment event logs are omitted; current requests and current deployment attributes are stored. Full configuration lineage and historical organizational restructuring are also deferred.

Clusters and service options: the model treats each resource as independently allocatable. Composite clusters, dependencies and shared-component topology are deferred to prevent double allocation. Services are selected at deployment time rather than reserved separately. Distinct priced options are separate SERVICE offerings. Workloads are described in deployment notes; workload-level inventory is not queried.

Contracts and pricing: SLA terms and qualification decisions can be documented in agreement and qualification notes. Formal contracts, funded deposits, payment history and eligibility calculations are excluded. Standard rate history is retained; time-of-day, duration, volume and bundle pricing require future pricing extensions. Staff can record authorized adjustments. Sharing is entered directly per charge; automated rules and defaults are deferred.

Access and operations: account kinds, memberships, confidentiality and staff actors provide metadata, but credentials, general audit logs and configurable grants are excluded. Staff approve resource/service compatibility manually. Autoscaling decisions, allocation optimization, fairness scoring and live data collection are external; their resulting allocations and current priorities are recorded. Interactive topology is deferred with the dependency model.

Payments and refunds: credits reduce billed charges; actual payments, cash refunds and settlement balances are not represented. Sample records use synthetic clients and contain no real credentials. These scope decisions account for omitted scenario details in the report, as permitted by the assignment, and remain subject to instructor feedback.

6. Validation example and team planning

Illustrative case: a team reserves two instances, each needing 2 vCPUs and 4 GiB of memory. Two matching 4-vCPU/8-GiB resources at capacity_fraction 0.5 fulfill the request. A failed assignment ends and a suitable replacement begins. Metered compute consumption generates a USD 60 charge. Staff allocate USD 36 and USD 24 to two billing accounts, and create one invoice per account. An authorized credit can compensate for a disruption.

The team statement assigns four proposed work packages and reciprocal reviewers. Members must confirm their names, assignments and acceptance. Milestone 1 includes only the ERD PDF, this draft and the team statement. Later work will implement the chosen course DBMS, generate at least a quarter of synthetic activity, and build aggregate queries and one or two read-only pages. Proposed queries cover utilization, shortages, downtime, substitutions, actual-versus-estimated use, client spending, payer shares and credits.

Source basis

Group Project - DB.pdf, Group Project_ CloudNine Systems.pdf, and the supplied Milestone 1 instructions. This 23-table core design supersedes the 30-table version; earlier versions remain in Git history.